

SSH

SSH (Secure Shell) ist ein Netzwerkprotokoll, das eine sichere Kommunikation zwischen einem Client und einem Server über ein unsicheres Netzwerk ermöglicht. Es wird häufig verwendet, um sich remote auf einem anderen Computer anzumelden und Befehle auszuführen, Dateien zu übertragen und Netzwerktunnel zu erstellen. SSH bietet eine verschlüsselte Verbindung, die die Vertraulichkeit und Integrität der übertragenen Daten gewährleistet. SSH verwendet in der Regel das TCP-Protokoll und standardmäßig den Port 22 für die Kommunikation. Es bietet verschiedene Authentifizierungsmethoden, darunter Passwortauthentifizierung und Public-Key-Authentifizierung, bei der ein Schlüsselpaar verwendet wird, um die Identität des Benutzers zu überprüfen. (vgl.)

Installation und erste Nutzung von SSH

Damit man SSH nutzen kann, muss der SSH-Server auf dem Zielrechner installiert und gestartet sein. Auf den meisten Distributionen kann der OpenSSH-Server mit dem Paketmanager installiert werden:

```
apt install openssh-server
```

TERMINAL

Nach der erfolgreichen Installation kann von einem anderen Rechner aus bereits eine Verbindung aufgebaut werden. Die Verbindung setzt hierbei im Standard eine Passwortauthentifizierung voraus:

```
ssh <User>@<IP-Adresse>
```

TERMINAL

Möchte man keine Verbindung sondern zum Beispiel lediglich Befehle ausführen, so kann dies natürlich auch durchgeführt werden:

```
ssh <User>@<IP-Adresse> <Befehl>
```

TERMINAL

Generierung von Schlüsselpaaren

Um nicht ständig mit Passwörtern arbeiten zu müssen, kann auch die Public-Key-Authentifizierung verwendet werden. Hierzu wird auf dem Client ein Schlüsselpaar generiert:

```
ssh-keygen
```

TERMINAL

Dieser Befehl erstellt zwei Dateien:

- `~/.ssh/id_rsa` : Dies ist der private Schlüssel, der geheim gehalten werden muss und niemals weitergegeben werden sollte.
- `~/.ssh/id_rsa.pub` : Dies ist der öffentliche Schlüssel, der auf den Server kopiert werden kann, um die Authentifizierung zu ermöglichen.

Um Einfluss auf den Namen des Schlüsselpaares zu nehmen, kann der Parameter `-f` (= file) verwendet werden:

```
ssh-keygen -f ~/.ssh/<Schlüsselname>
```

TERMINAL

Der Parameter `-t` (= `type`) ermöglicht es, den Typ des Schlüssels zu bestimmen. Mögliche Werte sind hierbei `rsa`, `dsa`, `ecdsa` und `ed25519`. Der Standardwert ist `rsa`.

```
ssh-keygen -t ed25519 -f ~/.ssh/<Schlüsselname>
```

TERMINAL

Der Parameter `-b` (= `bits`) bestimmt die Länge des Schlüssels. Je länger der Schlüssel, desto sicherer ist dieser. Der Standardwert ist hierbei 2048 für `rsa` und `dsa`, 256 für `ecdsa` und `ed25519`.

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/<Schlüsselname>
```

TERMINAL

Schlüssel auf den Server kopieren

Nachdem das Schlüsselpaar generiert wurde, muss der öffentliche Schlüssel auf den Server kopiert werden. Dies kann mit dem Befehl `ssh-copy-id` erfolgen:

```
ssh-copy-id -i ~/.ssh/<Schlüsselname>.pub <User>@<IP-Adresse>
```

TERMINAL

Wurde der Name des Schlüssels mit Hilfe des Parameters `-f` nicht verändert, kann der Parameter `-i` (= `identity file`) auch weggelassen werden:

```
ssh-copy-id <User>@<IP-Adresse>
```

TERMINAL

Nach der erfolgreichen Ausführung des Befehls wird der öffentliche Schlüssel zur Datei `~/.ssh/authorized_keys` auf dem Server hinzugefügt. Ab sofort kann die Verbindung zum Server ohne Passwort hergestellt werden!

NOTE

Sollte der Befehl `ssh-copy-id` nicht vorhanden sein (z. B. Windows), so kann der Schlüssel auch anderweitig auf den entfernten Rechner kopiert werden! Der Befehl hierfür sieht folgendermaßen aus:

```
cat ~/.ssh/<Schlüsselname>.pub | ssh <User>@<IP-Adresse> 'mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys'
```

TERMINAL

Dateien kopieren

Um Dateien zwischen dem lokalen Rechner und dem entfernten Rechner zu kopieren, kann der Befehl `scp` (Secure Copy) verwendet werden. `scp` nutzt SSH zur sicheren Übertragung der Dateien. Die generelle Syntax von `scp` ist:

```
scp <Quelle> <Ziel>
```

TERMINAL

Die Quelle und das Ziel können entweder lokale Pfade oder Remote-Pfade im Format `<User>@<IP-Adresse>:<Pfad>` sein. Um komplette Ordnerstrukturen rekursiv zu kopieren, wird der Parameter `-r` verwendet.

Ablauf der Verbindung

1. Der Client sendet eine Anfrage an den Port 22 des Servers.

2. Der Server antwortet mit dem öffentlichen Schlüssel, der für den User, der die Anfrage stellt, hinterlegt ist.
3. Der Client muss den öffentlichen Schlüssel überprüfen. Hierzu passieren folgende Dinge:
 - Beim ersten Verbindungsversuch zum Server wird der User gefragt, ob er dem Server vertraut und dessen Schlüssel akzeptieren möchte. Bejaht der User dies, wird der Schlüssel in die Datei `~/.ssh/known_hosts` auf dem Client gespeichert.
 - Bei weiteren Verbindungsversuchen wird der gespeicherte Schlüssel mit dem vom Server gesendeten Schlüssel verglichen. Stimmen diese überein, wird die Verbindung fortgesetzt. Andernfalls erhält der User eine Warnung, dass der Schlüssel nicht übereinstimmt (= MAN-IN-THE-MIDDLE-ATTACK).
4. Falls der Vorgang nicht abgebrochen wurde, wird vom Client ein symmetrischer Sessionkey generiert, der - verschlüsselt mit dem öffentlichen Schlüssel des Servers - zum Server gesendet wird.
5. Server als auch Client haben jetzt beide den symmetrischen Sessionkey, mit dem die Anmeldung und die komplette Datenübertragung verschlüsselt werden kann.

/etc/ssh/sshd_config

Bei dieser Datei handelt es sich um die Konfigurationsdatei des SSH-Servers. Hier können verschiedene Einstellungen vorgenommen werden, um den SSH-Zugriff zu steuern. Wichtige Parameter sind unter anderem:

- **Port** : Der Port, auf dem der SSH-Server lauscht (Standard: 22).
- **PermitRootLogin** : Legt fest, ob der Root-Login über SSH erlaubt ist. Der Standard ist hierbei `prohibit-password`, was bedeutet, dass der Root-Login nur mit einem öffentlichen Schlüssel und nicht mit einem Passwort erfolgen kann.
- **PasswordAuthentication** : Bestimmt, ob die Passwortauthentifizierung erlaubt ist (Standard: `yes`).
- **PubkeyAuthentication** : Legt fest, ob die Authentifizierung mit öffentlichen Schlüsseln erlaubt ist (Standard: `yes`).
- **AllowUsers** : Hier können spezifische Benutzer definiert werden, die sich per SSH anmelden dürfen.
- **PermitEmptyPasswords** : Bestimmt, ob Benutzer sich mit einem leeren Passwort anmelden dürfen (Standard: `no`).
- **DenyUsers** : Das Gegenteil von **AllowUsers**, wodurch Benutzer definiert werden können, die sich nicht via SSH anmelden dürfen.
- **AllowGroups** : Es können auch spezifische Gruppen eingetragen werden, für die der Zugriff via SSH erlaubt ist.

Um Änderungen an dieser Datei vorzunehmen, ist in der Regel Root-Zugriff erforderlich. Nach Änderungen muss der SSH-Dienst neu gestartet werden, damit die Änderungen wirksam werden.

SSH-Tunnel

Was ist ein SSH-Tunnel?

Sinn und Zweck von SSH ist es, dass man eine verschlüsselte Verbindung zu einem entfernten Rechner aufbaut und mit diesem so arbeiten kann, als würde man direkt vor dem Rechner sitzen. Ein **Tunnel** ist eine ähnliche Verbindung, wodurch Daten eines dritten Rechners, zum Beispiel einer Website, über einen SSH-Server verschlüsselt angefordert und bereitgestellt werden, wodurch ein weiterer Rechner darauf zugreifen kann. Ein Beispiel hierfür könnte sein, dass zum Beispiel der Netzwerkadministrator die Seite `example.com` gesperrt hat. Vom lokalen Rechner hat man Zugriff auf den SSH-Server `ssh.local.de`. Dieser SSH-Server hat eine Verbindung zu `example.com`. Der SSH-Tunnel

ermöglicht es nun, dass über den SSH-Server `ssh.local.de` die Seite `example.com` angefordert wird und an den lokalen Rechner weitergeleitet wird. Da das Ganze verschlüsselt geschieht, ist es nicht ersichtlich, welche Daten genau angesehen werden. (vgl. [HEI1])

Um dieses Prinzip zu verinnerlichen, wird die URL `https://edvschule-plattling.de (= 212.77.167.110:443)` verwendet. Um nun eine Verbindung vom `<User>_debian_client` aufbauen zu können, wird auf dem Client nachfolgender Befehl ausgeführt:

```
ssh <<User>@<IP-Adresse> -L <Port>:<Ziel>:<Zielport>
```

TERMINAL

Bei dem Befehl wird zunächst ganz klassisch eine SSH-Verbindung von einem User zu einer IP-Adresse aufgebaut. Über den Parameter `-L` wird ein lokaler Port am Rechner geöffnet, wodurch im Anschluss eine Weiterleitung auf eine gewisse Zieladresse mit einem Zielport durchgeführt wird. Gibt man nun am entsprechenden Rechner die URL `localhost:<Port>` ein, wird eine Verbindung über den SSH-Tunnel zur gewünschten Zieladresse und Zielport hergestellt! Bei diesem Prinzip handelt es sich um das `Local Port Forwarding` !.

Reverse SSH-Tunnel

Hierbei ist das Prinzip ähnlich zum normalen, lokalen SSH-Tunnel. Auf einem lokalen Rechner läuft eine Anwendung, die über das Internet generell nicht erreichbar ist. Die Anwendung sollte allerdings zur Verfügung gestellt werden. Hierfür kann nur ein Reverse SSH-Tunnel genutzt werden. Am entsprechenden Rechner muss nachfolgender Befehl ausgeführt werden:

```
ssh <User>@<IP-Adresse> -R <Port>:localhost:<Zielport>
```

TERMINAL

Dynamic SSH-Tunnel

Mit SSH ist man auch in der Lage einen eigenen `SOCKS5` Proxy zu erstellen. Ein Proxy selber ist dabei nichts anderes wie ein Vermittler, der Anfragen von Clients entgegennimmt und diese an einen anderen Server weiterleitet. Man kann somit auch seine eigene IP-Adresse verschleiern oder über den Proxy auch den Datenverkehr regeln und überwachen. `SOCKS5` hingegen ist ein Protokoll, das diese Funktionalität bereitstellt. Um einen entsprechenden `SOCKS5` Proxy zu erstellen, kann nachfolgender Befehl genutzt werden:

```
ssh <User>@<IP-Adresse> -D <LokalerPort>
```

TERMINAL

Im Beispiel wird sich folglich zu einem SSH-Server mit einer gewissen IP-Adresse und einem bestimmten User verbunden. Die Option `-D` ermöglicht es, einen lokalen `SOCKS5` Proxy zu erstellen, der dann für verschiedene Anwendungen genutzt werden kann. Die Angabe eines Ports ist hierfür zwingend erforderlich. Im Endeffekt wird im Anschluss jeder Datenverkehr, der an diesen lokalen Port gesendet wird, über den SSH-Tunnel an den SSH-Server weitergeleitet und von dort aus ins Internet geschickt. Dies ermöglicht es, die eigene IP-Adresse zu verschleiern und den Datenverkehr zu anonymisieren.